

Laporan Praktikum: Tutorial OpenMP

Pemrograman Paralel CPU dengan OpenMP dalam C

Aufa Akmal Bunaya

4 Oktober 2026

Ringkasan

Laporan ini mendokumentasikan pengerjaan delapan latihan tutorial OpenMP beserta mini-project perkalian matriks paralel. Seluruh program dikompilasi dengan GCC 13.3.0 memakai flag `-fopenmp` dan dijalankan secara nyata pada VM Linux 2 vCPU. Setiap latihan diverifikasi kebenarannya, modifikasi yang diminta tutorial diimplementasikan dan diuji, serta pertanyaan investigasi dijawab berdasarkan perilaku yang teramati. Latihan 8 mengukur speedup riil: pada $n = 10^8$ iterasi, 2 thread mencapai speedup 1,53 (efisiensi 0,76) dan 4 thread mencapai speedup 1,68 (efisiensi 0,42, oversubscription pada 2 vCPU), dengan selisih numerik serial-paralel $7,4 \times 10^{-13}$ di bawah toleransi 10^{-8} . Mini-project memverifikasi perkalian matriks 256×256 : versi paralel identik dengan serial (selisih 0,0) dan mencapai speedup 2,00 pada 2 thread.

Daftar Isi

1	Lingkungan	2
2	Exercise 1: Membuat tim thread	2
3	Exercise 2: Data shared dan private	3
4	Exercise 3: Paralelisasi penjumlahan vektor	4
5	Exercise 4: Akumulasi aman dengan reduction	4
6	Exercise 5: Perbandingan loop schedule	5
7	Exercise 6: Koordinasi fase dependen	6
8	Exercise 7: Membuat task independen	6
9	Exercise 8: Kebenaran numerik dan speedup	7
9.1	Tabel ringkasan (median 5 run)	7
9.2	Validasi numerik	7
9.3	Penjelasan skala dan jawaban pertanyaan akhir	7
10	Mini-project: Perkalian matriks paralel	9
11	Refleksi: satu bug dan diagnosisnya	10
12	Reproduksi	10

1 Lingkungan

Komponen	Spesifikasi
Mesin	VM <code>htch-runtime</code> , Ubuntu 24.04.5 LTS
CPU	AMD EPYC 9D25 126-Core, 2 vCPU tersedia
RAM	7,7 GiB
Kompiler	GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Flag	<code>-std=c11 -O2 -Wall -Wextra -fopenmp</code> (link: <code>-lm</code> bila memakai <code>math.h</code>)
Variabel	<code>OMP_DYNAMIC=FALSE</code> ; jumlah thread lewat <code>OMP_NUM_THREADS</code>
Jumlah thread uji	1, 2, 4 (4 thread = oversubscription pada 2 vCPU, disengaja untuk diamati)

```
Environment - GCC with OpenMP

student@htch-runtime:~/openmp-lab$ gcc --version | head -1
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
student@htch-runtime:~/openmp-lab$ nproc && grep -m1 'model name' /proc/cpuinfo
2
model name: AMD EPYC 9D25 126-Core Processor
student@htch-runtime:~/openmp-lab$ gcc -std=c11 -O2 -Wall -Wextra -fopenmp src/hello_openmp.c -o bin/hello
BUILD_OK
```

Gambar 1: Pemeriksaan lingkungan: versi GCC, jumlah vCPU, dan kompilasi pertama dengan `-fopenmp`.

Seluruh program dibangun dengan satu perintah `make` memakai Makefile pada bagian 12. Tidak ada warning kompilasi pada 14 program.

2 Exercise 1: Membuat tim thread

Berkas: `src/hello_openmp.c`. Program mencetak ukuran tim sebelum dan sesudah region paralel, dan setiap thread mencetak sapaan di dalam `#pragma omp critical`.

```
Exercise 1 - thread team (4 threads, two runs)

student@htch-runtime:~/openmp-lab$ OMP_DYNAMIC=FALSE OMP_NUM_THREADS=4 ./bin/hello_openmp
Before: team size = 1
Hello from thread 1 of 4
Hello from thread 0 of 4
Hello from thread 2 of 4
Hello from thread 3 of 4
After: team size = 1
student@htch-runtime:~/openmp-lab$ OMP_DYNAMIC=FALSE OMP_NUM_THREADS=4 ./bin/hello_openmp
Before: team size = 1
Hello from thread 2 of 4
Hello from thread 0 of 4
Hello from thread 3 of 4
Hello from thread 1 of 4
After: team size = 1
```

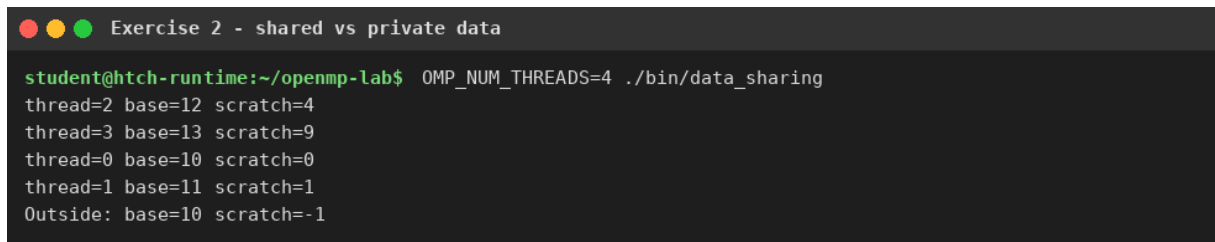
Gambar 2: Dua eksekusi dengan 4 thread: urutan sapaan berbeda antar run, tetapi setiap thread tepat mencetak satu baris.

Hasil investigasi:

1. Thread yang diminta vs aktual: `OMP_NUM_THREADS=1, 2, 4` semuanya dipenuhi persis (`team size` aktual = 1, 2, 4).
2. Urutan sapaan berubah antar run (tiga run 4-thread menghasilkan tiga urutan berbeda) karena penjadwalan thread oleh OS tidak deterministik; `critical` hanya menyerikan akses, tidak mengurutkan ID thread.
3. `printf` sebelum region dieksekusi tepat satu kali (oleh thread awal), sedangkan di dalam region dieksekusi sekali per thread — region paralel dieksekusi penuh oleh setiap anggota tim.
4. `After` selalu muncul terakhir karena akhir region paralel memiliki barrier implisit: thread awal baru melanjutkan setelah seluruh tim tiba dan tersinkronisasi.

3 Exercise 2: Data shared dan private

Berkas: `src/data_sharing.c`, memakai `#pragma omp parallel default(none) firstprivate(base) private(scratch)`.



```

Exercise 2 - shared vs private data
student@htch-runtime:~/openmp-lab$ OMP_NUM_THREADS=4 ./bin/data_sharing
thread=2 base=12 scratch=4
thread=3 base=13 scratch=9
thread=0 base=10 scratch=0
thread=1 base=11 scratch=1
Outside: base=10 scratch=-1

```

Gambar 3: Setiap thread mendapat salinan privat `base=10+t` dan `scratch=t2`; variabel asli di luar region tidak berubah.

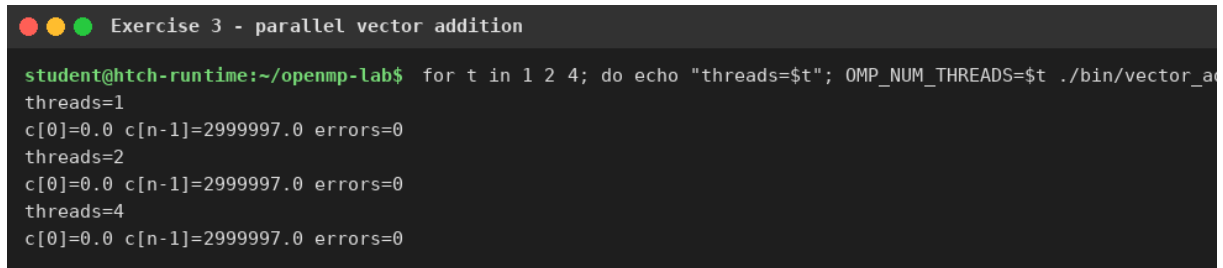
Hasil investigasi:

1. Perubahan `base` di dalam region tidak memengaruhi `base` asli (`Outside: base=10`) karena `firstprivate` memberi tiap thread *salinan privat* yang diinisialisasi dengan nilai 10.
2. Mengganti `private(scratch)` menjadi `firstprivate(scratch)` lalu mencetak nilainya sebelum assignment: setiap thread mengamati nilai awal `-1` (salinan dari nilai asli). Ini membuktikan `firstprivate` menyalin nilai masuk, sedangkan `private` tidak.
3. Membaca `private(scratch)` sebelum assignment pertama tidak valid: variabel privat tidak diinisialisasi, nilainya tak terdefinisi — nilai “kebetulan” yang tercetak tidak boleh dianggap bermakna.
4. Menghapus `firstprivate(base)` dengan `default(none)` menggagalkan kompilasi: `error: 'base' not specified in enclosing 'parallel'`. Klausula `default(none)` memaksa setiap variabel yang dipakai di dalam region dideklarasikan atribut data-sharing-nya secara eksplisit.

Catatan: `firstprivate` pada sebuah *pointer* hanya menyalin alamatnya; seluruh thread tetap menunjuk memori yang sama. Kepemilikan memori harus diputuskan terpisah dari kepemilikan variabel pointer.

4 Exercise 3: Paralelisasi penjumlahan vektor

Berkas: `src/vector_add.c` ($n = 10^6$), memakai `#pragma omp parallel for default(none) shared(a,b,c,n) schedule(static)`. Penyimpanan array bersifat `shared`; variabel loop `i` privat menurut aturan worksharing-loop OpenMP.



```
Exercise 3 - parallel vector addition
student@htch-runtime:~/openmp-lab$ for t in 1 2 4; do echo "threads=$t"; OMP_NUM_THREADS=$t ./bin/vector_add
threads=1
c[0]=0.0 c[n-1]=2999997.0 errors=0
threads=2
c[0]=0.0 c[n-1]=2999997.0 errors=0
threads=4
c[0]=0.0 c[n-1]=2999997.0 errors=0
```

Gambar 4: Hasil identik (`errors=0`) pada 1, 2, dan 4 thread.

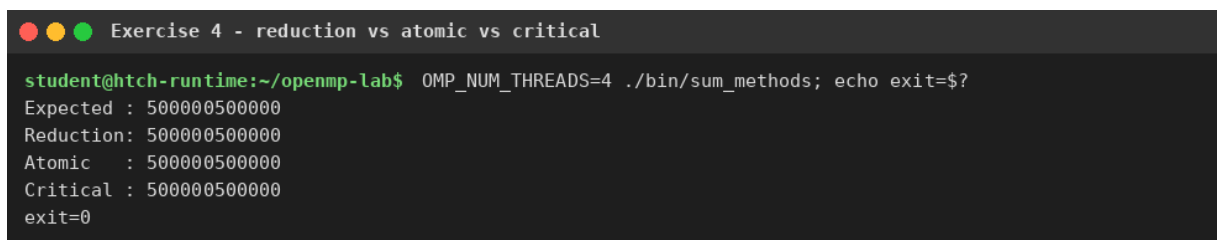
Modifikasi yang diminta dan hasilnya:

1. Terverifikasi pada 1, 2, 4 thread: `c[0]=0.0 c[n-1]=2999997.0 errors=0`.
2. $c_i = \alpha a_i + b_i$ dengan $\alpha = 2$ (`src/vector_add_alpha.c`, `alpha` ditambahkan ke klausa `shared`): `c[n-1]=3999996.0 errors=0`, sesuai harapan $4i$.
3. Bentuk terpisah (`src/vector_add_separate.c`: `#pragma omp parallel` + `#pragma omp for` di dalamnya) menghasilkan nilai numerik yang sama persis. Bentuk ini berguna ketika beberapa operasi worksharing memakai ulang tim yang sama.
4. Jika baris `omp for` dihapus dari struktur terpisah, *setiap* thread mengeksekusi seluruh loop: terjadi data race pada `c` karena beberapa thread menulis lokasi memori yang sama tanpa sinkronisasi. Penulisan konkuren yang “kebetulan identik” tetap merupakan data race — model memori tidak menjamin hasilnya.

Pemeriksaan ketergantungan: loop seperti `a[i] = a[i-1] + b[i]` memiliki dependensi antar iterasi yang berdekatan; menambah `parallel for` tidak menghilangkan dependensi tersebut.

5 Exercise 4: Akumulasi aman dengan reduction

Berkas: `src/sum_methods.c` menjumlah $1 \dots 10^6$ (ekspektasi 500000500000) dengan tiga metode yang benar.



```
Exercise 4 - reduction vs atomic vs critical
student@htch-runtime:~/openmp-lab$ OMP_NUM_THREADS=4 ./bin/sum_methods; echo exit=$?
Expected : 500000500000
Reduction : 500000500000
Atomic : 500000500000
Critical : 500000500000
exit=0
```

Gambar 5: Ketiga metode menghasilkan nilai yang tepat pada 4 thread.

Hasil investigasi:

1. Ketiga metode benar pada 1, 2, dan 4 thread (`exit=0`).

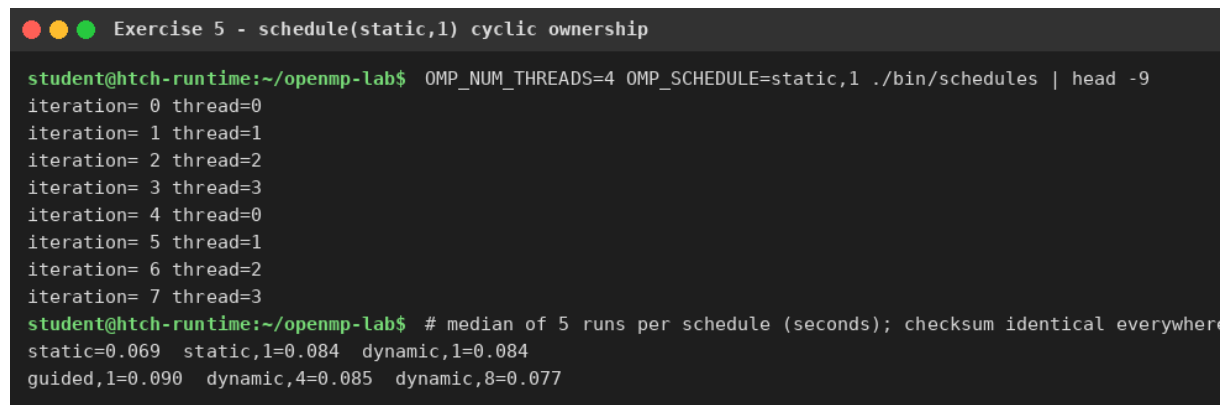
2. `sum_reduction` tidak boleh sekaligus tercantum di `shared(...)` pada direktif yang sama: klausa `reduction` sudah memberi setiap partisipan akumulator privat yang digabungkan di akhir; variabel yang sama tidak dapat berstatus `shared` dan `reduction` sekaligus.
3. Jumlah kuadrat $1 \dots 1000$ dengan `reduction` (`src/sum_squares.c`): `expected=333833500` `got=333833500` OK, sesuai rumus $n(n+1)(2n+1)/6$.
4. Pengukuran waktu ketiga metode ($n = 10^7$, 5 repetisi, 4 thread, tanpa pencetakan di dalam timer):

Metode	Waktu/run (s)	Relatif terhadap <code>reduction</code>
<code>reduction</code>	0,002970	1×
<code>atomic update</code>	0,230224	78×
<code>critical</code>	0,508649	171×

Reduction jauh lebih cepat karena tiap thread mengakumulasi secara lokal tanpa sinkronisasi per iterasi dan hanya digabungkan sekali di akhir; `atomic` dan `critical` menyerikan setiap penambahan sehingga membatasi konkurensi. Region `critical` yang melindungi invarian yang sama harus memakai nama yang sama.

6 Exercise 5: Perbandingan loop schedule

Berkas: `src/schedules.c` ($N = 32$, beban kerja tiap iterasi tumbuh dengan i), memakai `schedule(runtime)` sehingga kebijakan dipilih lewat `OMP_SCHEDULE`.



```

Exercise 5 - schedule(static,1) cyclic ownership
student@htch-runtime:~/openmp-lab$ OMP_NUM_THREADS=4 OMP_SCHEDULE=static,1 ./bin/schedules | head -9
iteration= 0 thread=0
iteration= 1 thread=1
iteration= 2 thread=2
iteration= 3 thread=3
iteration= 4 thread=0
iteration= 5 thread=1
iteration= 6 thread=2
iteration= 7 thread=3
student@htch-runtime:~/openmp-lab$ # median of 5 runs per schedule (seconds); checksum identical everywhere
static=0.069 static,1=0.084 dynamic,1=0.084
guided,1=0.090 dynamic,4=0.085 dynamic,8=0.077

```

Gambar 6: Pola kepemilikan siklik `schedule(static,1)`: iterasi i ditangani thread $i \bmod 4$; median 5 run per kebijakan.

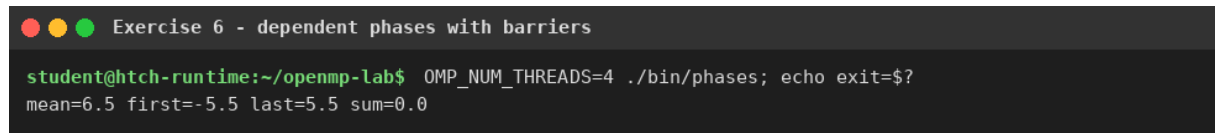
Hasil investigasi:

1. Pola `static,1` menunjukkan penugasan siklik chunk berukuran 1 (iterasi 0–3 ke thread 0–3, lalu berulang), persis seperti teori.
2. Checksum identik untuk semua kebijakan (`checksum=468.442495904457`): penjadwalan hanya mengubah *siapa* mengerjakan iterasi, bukan hasil tiap iterasi.
3. Median 5 run (detik): `static` 0,069; `static,1` 0,084; `dynamic,1` 0,084; `guided,1` 0,090; `dynamic,4` 0,085; `dynamic,8` 0,077. Pada mesin dan ukuran masalah ini tidak ada kebijakan yang dominan — run terlalu singkat ($< 0,1$ s) sehingga noise VM menutupi perbedaan; sesuai peringatan tutorial, `dynamic` tidak otomatis lebih cepat.

4. Chunk dynamic 1, 4, 8: chunk kecil mendistribusikan beban tak-merata lebih halus tetapi menambah overhead penjadwalan; chunk besar sebaliknya. Pada eksperimen ini perbedaannya tertutup noise, yang justru menegaskan bahwa klaim kecepatan harus diukur, bukan disimpulkan dari ID thread.

7 Exercise 6: Koordinasi fase dependen

Berkas: `src/phases.c`: inialisasi vektor $\rightarrow$ hitung rata-rata sekali $\rightarrow$ pusatkan setiap elemen. Keluaran sesuai harapan: `mean=6.5 first=-5.5 last=5.5 sum=0.0`.



```
Exercise 6 - dependent phases with barriers
student@htch-runtime:~/openmp-lab$ OMP_NUM_THREADS=4 ./bin/phases; echo exit=$?
mean=6.5 first=-5.5 last=5.5 sum=0.0
```

Gambar 7: Hasil exercise 6 sesuai nilai harapan.

Hasil investigasi:

1. Barrier implisit pertama (akhir `for` inialisasi) melindungi dependensi tulis-baca: memastikan seluruh `a[i]` sudah terisi sebelum blok `single` menjumlahkannya. Barrier implisit kedua (akhir `single`) memastikan `mean` sudah ditulis sebelum loop pemusatan membacanya.
2. Varian `nowait` + `#pragma omp barrier` eksplisit sebelum `single` (`src/phases_nowait.c`) tetap benar (`mean=6.5 ... sum=0.0, exit=0`).
3. Menghapus barrier eksplisit itu tidak aman: barrier di akhir `single` tidak dapat melindungi pembacaan `a` yang terjadi *sebelumnya*.
4. Menambah `nowait` pada `single` tidak aman kecuali ada sinkronisasi lain: thread dapat lanjut ke loop pemusatan dan membaca `mean` sebelum thread pelaksana `single` selesai menulisnya.
5. Setiap anggota tim harus mencapai barrier tim; barrier di dalam kondisi yang hanya dimasuki sebagian thread (mis. `if (omp_get_thread_num() == 0)`) membuat program invalid dan dapat hang.

8 Exercise 7: Membuat task independen

Berkas: `src/tasks.c`: satu thread (`single`) membangkitkan 8 task (i^2), tiap task memakai `firstprivate(i)`, lalu `taskwait` sebelum hasil dicetak dan divalidasi.

Hasil investigasi:

1. Ya: pada run 4-thread, seluruh 8 task dieksekusi thread 3. Jumlah task dan jumlah thread adalah besaran yang berbeda; thread mengambil task dari pool secara dinamis.
2. `single` memastikan himpunan task dibangkitkan tepat sekali; `firstprivate(i)` mengabadikan indeks iterasi saat task dibuat sehingga tetap benar walau task dieksekusi belakangan; `taskwait` menunggu task-task anak selesai sebelum hasil dibaca (ia bukan barrier tim umum).
3. Memindah loop pencetakan sebelum `taskwait` tidak aman: hasil belum tentu siap karena task dapat dieksekusi setelah pencetakan dimulai.
4. Granularitas penting: task se-kecil i^2 ini hanya untuk demonstrasi semantik; biaya pembuatan task biasanya melebihi biaya aritmetikanya.

```

Exercise 7 - tasks executed by the team

student@htch-runtime:~/openmp-lab$ OMP_NUM_THREADS=4 ./bin/tasks; echo exit=$?
task=0 result=0 executed_by=3
task=1 result=1 executed_by=3
task=2 result=4 executed_by=3
task=3 result=9 executed_by=3
task=4 result=16 executed_by=3
task=5 result=25 executed_by=3
task=6 result=36 executed_by=3
task=7 result=49 executed_by=3
exit=0

```

Gambar 8: Dengan 4 thread, kedelapan task dieksekusi oleh thread 3 — satu thread boleh mengeksekusi banyak task.

9 Exercise 8: Kebenaran numerik dan speedup

Berkas: `src/pi_benchmark.c` mengaproksimasi π dengan aturan titik tengah, $n = 10^8$. Prosedur: biner dan n sama untuk semua konfigurasi, `OMP_DYNAMIC=FALSE`, 5 run per jumlah thread, timer hanya mencakup interval paralel (termasuk pembentukan tim, loop, sinkronisasi, reduction; warm-up dan I/O dikecualikan).

```

Exercise 8 - pi benchmark (sample run, 2 threads)

student@htch-runtime:~/openmp-lab$ OMP_DYNAMIC=FALSE OMP_NUM_THREADS=2 ./bin/pi_benchmark; echo exit=$?
n=100000000 warmup_threads=2 actual_threads=2
serial_pi=3.141592653590 parallel_pi=3.141592653590
difference=5.165e-13 reference_error=1.168e-13
serial_seconds=0.214234 parallel_seconds=0.118151
speedup=1.813 efficiency=0.907
exit=0
student@htch-runtime:~/openmp-lab$ # median of 5 runs: p=1 Sp=1.057 | p=2 Sp=1.527 Ep=0.764 | p=4 Sp=1.683

```

Gambar 9: Sampel run 2 thread: kedua estimasi π cocok hingga 12 desimal, selisih jauh di bawah toleransi.

9.1 Tabel ringkasan (median 5 run)

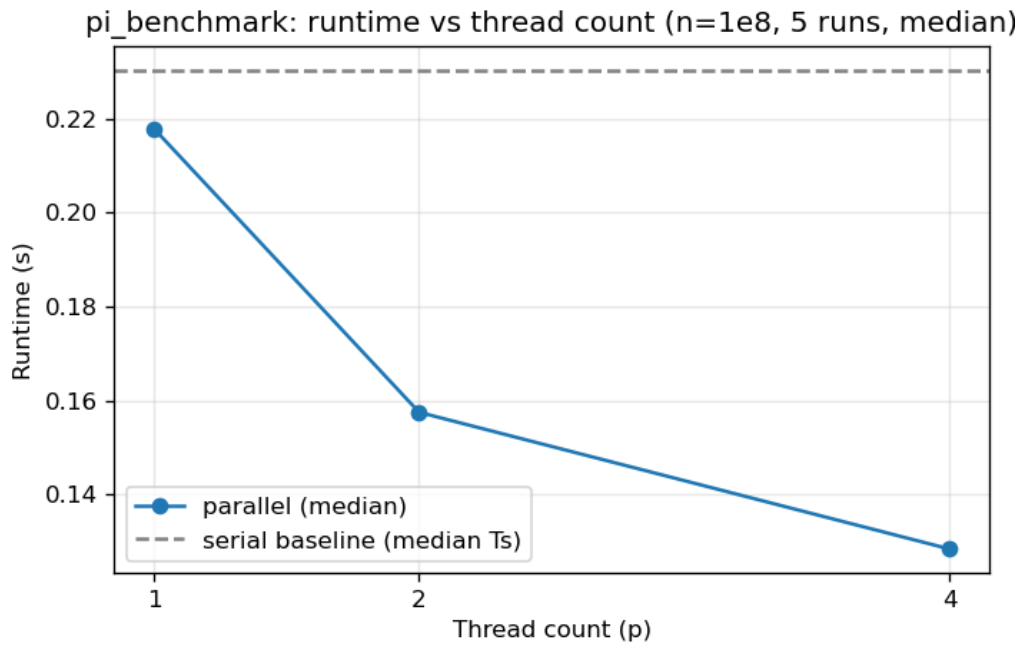
p aktual	Median T_s (s)	Median T_p (s)	S_p	E_p	Run
1	0,230283	0,217914	1,057	1,057	5
2	0,240557	0,157522	1,527	0,764	5
4	0,216187	0,128416	1,683	0,421	5

9.2 Validasi numerik

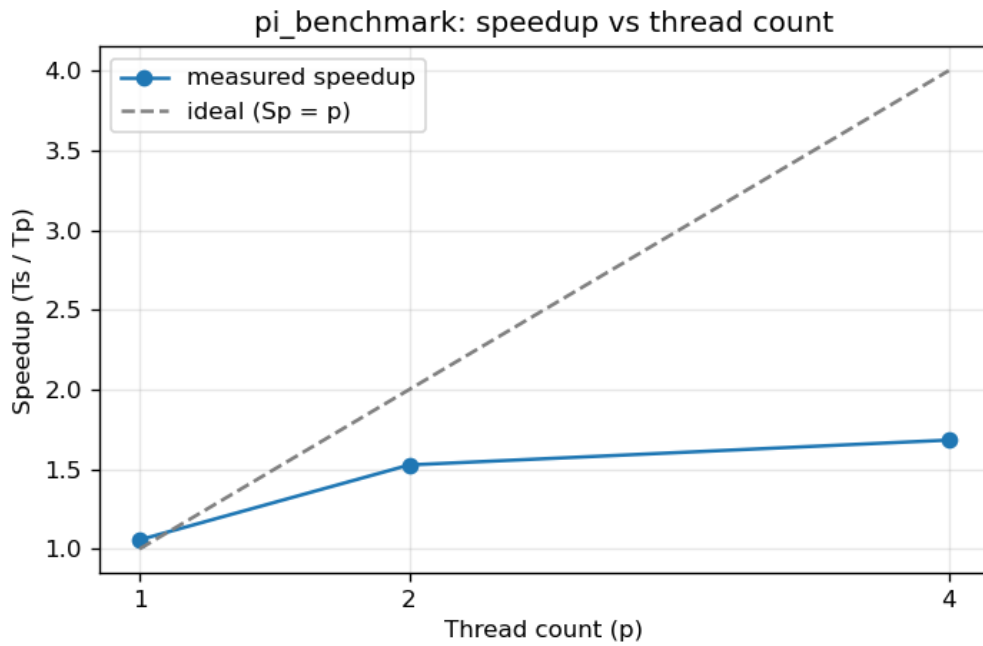
Selisih serial-paralel maksimum $7,438 \times 10^{-13} < 10^{-8}$ (lolos, `exit=0` pada seluruh 15 run); `reference_error` maksimum $6,333 \times 10^{-13}$ terhadap `acos(-1)`. Kedua implementasi cocok dan akurat.

9.3 Penjelasan skala dan jawaban pertanyaan akhir

1. Tercepat pada 4 thread ($S_p = 1,68$), tetapi efisiensinya hanya 0,42: 4 thread pada 2 vCPU adalah oversubscription. Skala “jujur” terlihat pada 2 thread: $S_p = 1,53$, $E_p = 0,76$.



Gambar 10: Runtime median vs jumlah thread ($n = 10^8$); garis putus-putus = baseline serial.



Gambar 11: Speedup terukur vs jumlah thread; garis putus-putus = speedup ideal $S_p = p$.

2. Satu thread OpenMP (0,218 s) tidak persis sama dengan loop serial (0,230 s): region OpenMP satu thread tetap menanggung overhead pembentukan tim dan reduction, walau di sini warm-up membuatnya sedikit lebih cepat — keduanya baseline yang berbeda dan tidak dapat dipertukarkan.
3. Memperbesar ukuran masalah membuat paralelisme lebih berguna: overhead tetap (pembentukan tim, sinkronisasi) teramortisasi oleh porsi kerja paralel yang lebih besar.
4. Pengukuran tambahan yang membantu menjelaskan perlambatan: pinning thread (OMP_PROC_BIND), waktu per thread, bandwidth memori, frekuensi CPU selama run, dan variasi urutan run untuk mendeteksi interferensi VM.

Batas Amdahl $S_p \leq 1/(f + (1 - f)/p)$ adalah batas ideal dengan kerja tetap; program riil juga menghadapi batas bandwidth memori, ketidakseimbangan beban, dan interferensi VM — thread lebih banyak tidak harus berarti lebih cepat.

10 Mini-project: Perkalian matriks paralel

Berkas: `src/matmul.c` mengimplementasikan $C = AB$ untuk matriks $N \times N$ padat dalam array 1D row-major, alokasi heap, dalam tiga varian: serial, `#pragma omp parallel for schedule(static)` pada loop i , dan varian `collapse(2)` pada loop i, j . Akumulasi memakai variabel lokal per iterasi sehingga tidak perlu reduction antar thread.

```

Mini-project - parallel matrix multiplication (n=256)
student@htch-runtime:~/openmp-lab$ OMP_NUM_THREADS=2 ./bin/matmul 256; echo exit=?
n=256 threads=2
identity_check: serial_vs_A=0.000e+00 parallel_vs_A=0.000e+00
verify: parallel_vs_serial=0.000e+00 collapse_vs_serial=0.000e+00 tol=1e-09 OK
time_serial=0.043495 time_parallel=0.021731 time_collapse=0.026347
speedup_parallel=2.001 speedup_collapse=1.651
exit=0

```

Gambar 12: Verifikasi $n = 256$, 2 thread: cek identitas tepat, paralel identik dengan serial, speedup 2,00.

Hasil ($n = 256$, waktu hanya perkalian):

Thread	Serial (s)	Paralel (s)	Collapse(2) (s)	Speedup par./coll.
1	0,0488	0,0698	0,0428	0,70 / 1,14
2	0,0435	0,0217	0,0263	2,00 / 1,65
4	0,0422	0,0233	0,0214	1,81 / 1,98

1. Cek identitas ($B = I$): $C = A$ tepat (0.000e+00) untuk serial maupun paralel.
2. Nilai nontrivial deterministik: paralel dan collapse(2) identik dengan serial (0.000e+00 < toleransi 10^{-9}) pada seluruh konfigurasi.
3. `collapse(2)` mendistribusikan ruang iterasi gabungan (i, j); setiap keluaran tetap memiliki akumulator lokalnya sendiri. Pada $n = 256$, collapse(2) sedikit lebih baik pada 4 thread (1,98 vs 1,81) karena pembagian kerja lebih halus.
4. $n = 128$ terlalu kecil (≈ 3 ms) sehingga hasilnya didominasi noise — pembandingan tetap memakai baseline serial yang setara (urutan loop dan flag identik), sesuai prinsip tutorial.

11 Refleksi: satu bug dan diagnosisnya

Bug yang ditemui: program mini-project `matmul.c` gagal dikompilasi dengan galat

```
1 src/matmul.c:107:50: error: expected '#pragma omp' clause before '{'
   token
```

pada baris `#pragma omp parallel default(none) shared(n) { (void)n; }`. Diagnosis dilakukan dengan program uji minimal: `pragma` yang diikuti blok `{` pada *baris berikutnya* dikompilasi dengan baik, sedangkan `{` pada *baris yang sama* dengan `pragma` selalu gagal. Ternyata parser OpenMP GCC tidak menerima kurung kurawal blok terstruktur pada baris yang sama dengan direktif `#pragma omp parallel` — blok terstruktur harus menjadi unit sintaksis tersendiri setelah direktif. Perbaikan: pindahkan `{` ke baris berikutnya; kompilasi kemudian bersih tanpa warning. Bug ini tidak terkait semantik paralelisme sama sekali, melainkan aturan penulisan direktif — pengingat bahwa pesan galat kompiler OpenMP perlu dibaca harfiah per baris.

12 Reproduksi

Seluruh sumber, Makefile, bukti output mentah (*evidence/*), screenshot, dan plot tersedia pada repositori: <https://github.com/aufaakmalbunaya/openmp-tutorial>. Perintah reproduksi persis:

```
1 make
2 export OMP_DYNAMIC=FALSE
3 OMP_NUM_THREADS=4 ./bin/hello_openmp
4 OMP_NUM_THREADS=4 ./bin/data_sharing
5 for t in 1 2 4; do OMP_NUM_THREADS=$t ./bin/vector_add; done
6 OMP_NUM_THREADS=4 ./bin/sum_methods
7 for s in static "static,1" "dynamic,1" "guided,1"; do \
8     OMP_NUM_THREADS=4 OMP_SCHEDULE="$s" ./bin/schedules; done
9 OMP_NUM_THREADS=4 ./bin/phases
10 OMP_NUM_THREADS=4 ./bin/tasks
11 for t in 1 2 4; do OMP_NUM_THREADS=$t ./bin/pi_benchmark; done
12 OMP_NUM_THREADS=2 ./bin/matmul 256
```

Laporan ini dikompilasi dari `latex_report/laporan_openmp.tex` dengan pdfLaTeX (dua pass).

Referensi

Tutorial “OpenMP Programming” (22 halaman, edisi bahasa Inggris) beserta dokumentasi GCC OpenMP dan spesifikasi OpenMP 5.1/5.2 yang dirujuk di dalamnya.